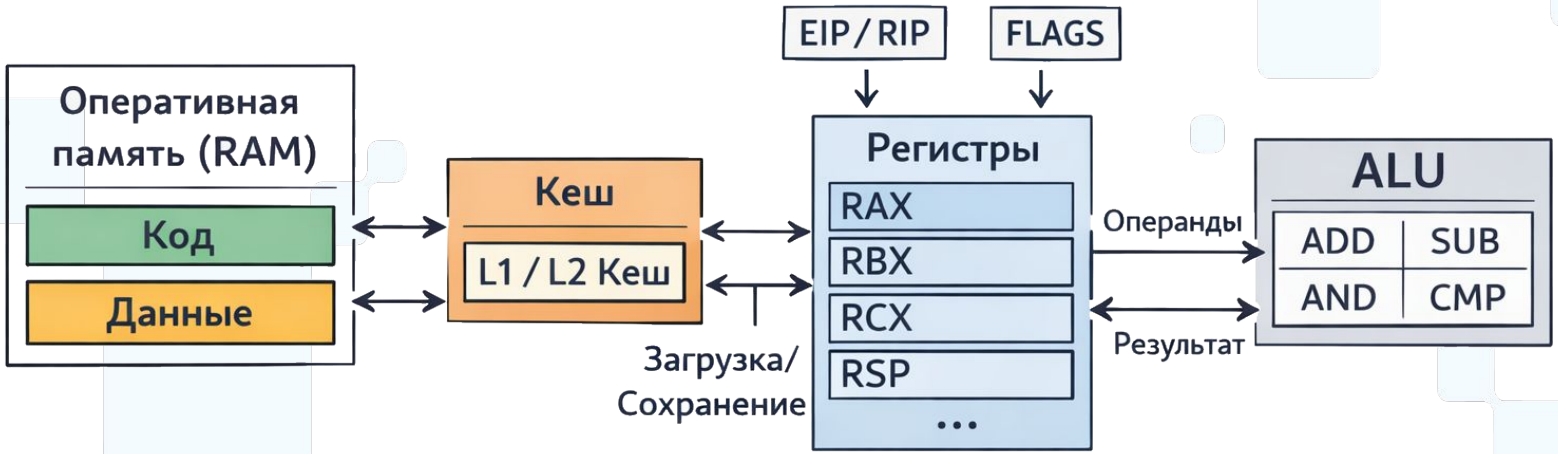
FAST FORWARD

АКОС

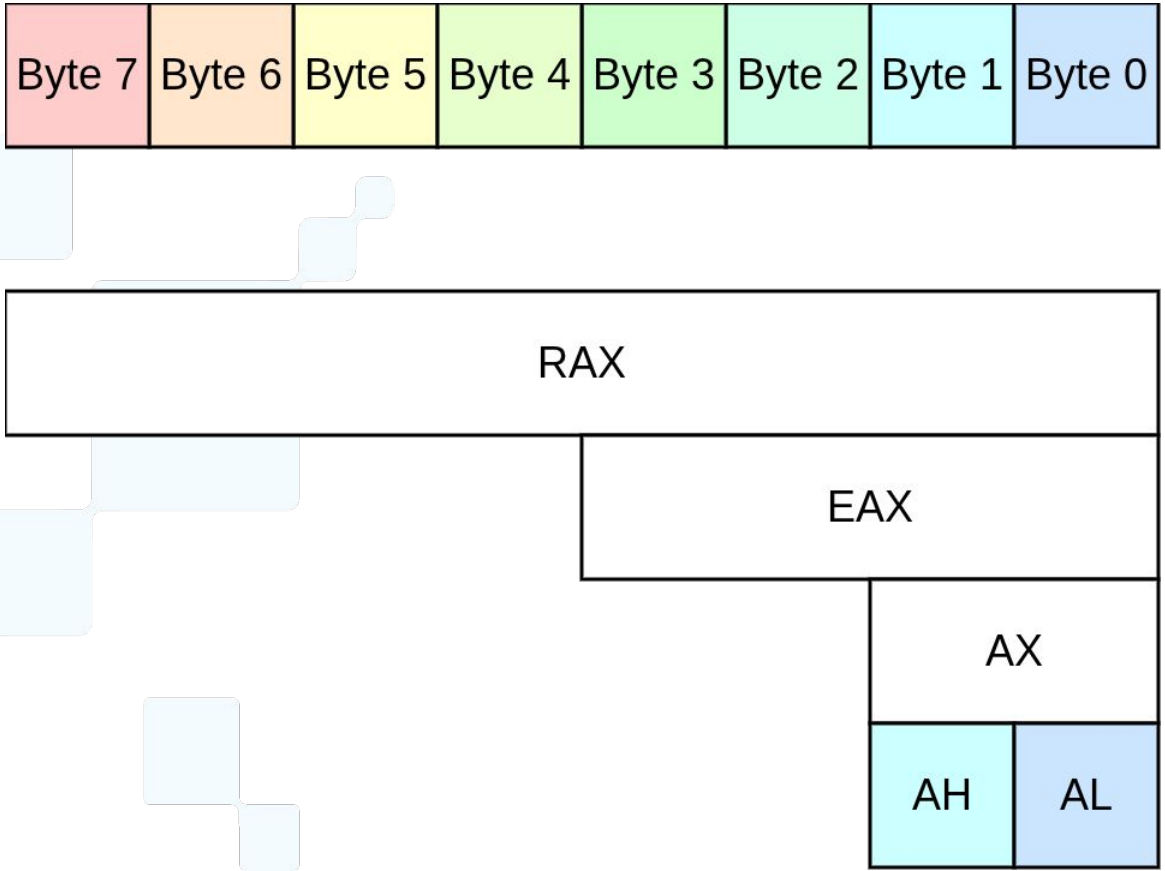
Лекция 3
X86_64 часть 2

FAST
FORWARD

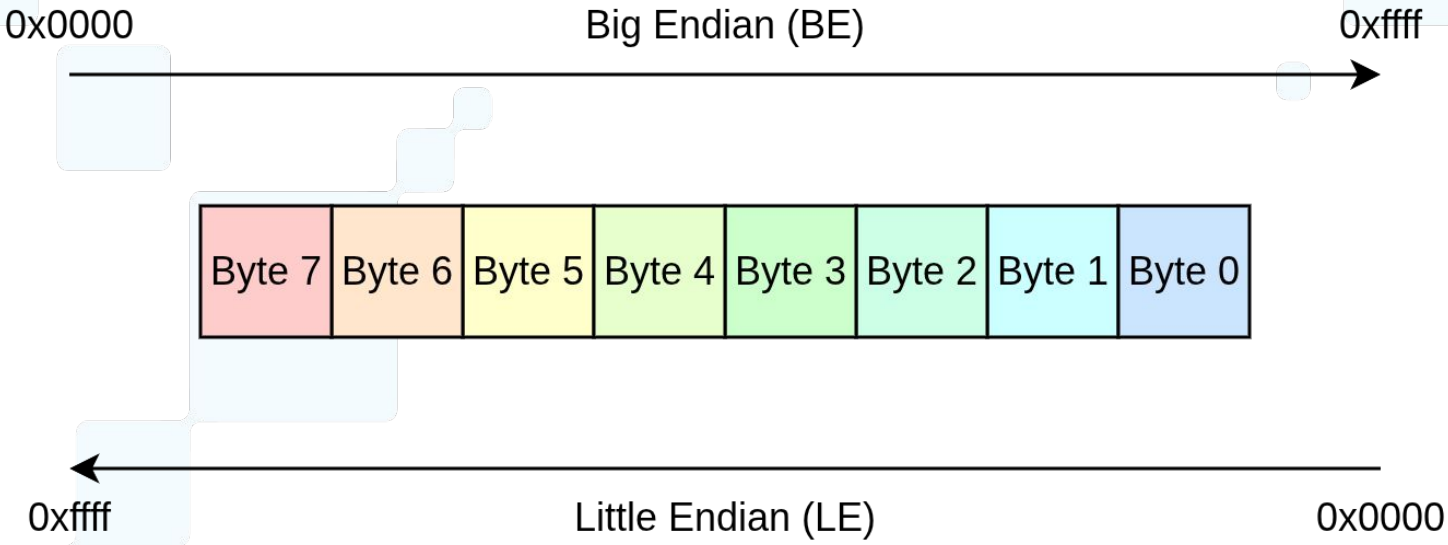
X86



FAST
FORWARD



FAST
FORWARD



FAST FORWARD

0x0000 0x0102030405060708 0xffff

Big Endian (BE)

Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0
01	02	03	04	05	06	07	08

Little Endian (LE)

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
08	07	06	05	04	03	02	01

FAST FORWARD

0x0000 0x0102030405060708 0xffff

Big Endian (BE) Сеть, MIPS, IBM Sys

Byte 7 01	Byte 6 02	Byte 5 03	Byte 4 04	Byte 3 05	Byte 2 06	Byte 1 07	Byte 0 08

Little Endian (LE) X86, ARM, RISC-V

Byte 0 08	Byte 1 07	Byte 2 06	Byte 3 05	Byte 4 04	Byte 5 03	Byte 6 02	Byte 7 01

FAST FORWARD

0x0000 0x0102030405060708 0xffff

Big Endian (BE)

Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0
01	02	03	04	05	06	07	08

Little Endian (LE)

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
08	07	06	05	04	03	02	01

MOV

```
mov DST, SRC    // копировать SRC в DST

mov ebx, eax

mov ebx, 0x1020
```

MOV

```
mov DST, SRC    // копировать SRC в DST

mov [0x40100], eax

mov ebx, [0x40100]

mov dword ptr [0x40100], 0x1543
```

FAST FORWARD

MOV

```
mov DST, SRC    // копировать SRC в DST

mov dword ptr [0x40100], eax

mov ebx, dword ptr [0x40100]

mov dword ptr [0x40100], 0x1543
```

MOV

mov DST, SRC // копировать SRC в DST

mov ebx, [0x40100] 8b 1c 25 **00 01 04 00**

mov dword ptr [0x40100], 0x1543 c7 04 25 **00 01 04 00**

43 15 00 00

MOV

mov DST, SRC // копировать SRC в DST

mov %eax, [0x102030405060708]

a3 08 07 06 05 04 03 02 01

MOV

```
mov DST, SRC     // копировать SRC в DST

mov dword ptr [0x0102030405060708], 0x0102030405060708
```

FAST FORWARD

MOV

Opcode	Instruction	Description
88 /r	MOV r/m8, r8	Move r8 to r/m8.
89 /r	MOV r/m16, r16	Move r16 to r/m16.
89 /r	MOV r/m32, r32	Move r32 to r/m32.
REX.W + 89 /r	MOV r/m64, r64	Move r64 to r/m64.
8A /r	MOV r8, r/m8	Move r/m8 to r8.
8B /r	MOV r16, r/m16	Move r/m16 to r16.
8B /r	MOV r32, r/m32	Move r/m32 to r32.
REX.W + 8B /r	MOV r64, r/m64	Move r/m64 to r64.

MOV

Opcode	Instruction	Description
B0+ rb ib	MOV r8, imm8	Move imm8 to r8.
B8+ rw iw	MOV r16, imm16	Move imm16 to r16.
B8+ rd id	MOV r32, imm32	Move imm32 to r32.
REX.W + B8+ rd io	MOV r64, imm64	Move imm64 to r64.
C6 /0 ib	MOV r/m8, imm8	Move imm8 to r/m8.
C7 /0 iw	MOV r/m16, imm16	Move imm16 to r/m16.
C7 /0 id	MOV r/m32, imm32	Move imm32 to r/m32.
REX.W + C7 /0 id	MOV r/m64, imm32	Move imm32 sign extended to 64-bits to r/m64.

MOV

mov DST, SRC // копировать SRC в DST

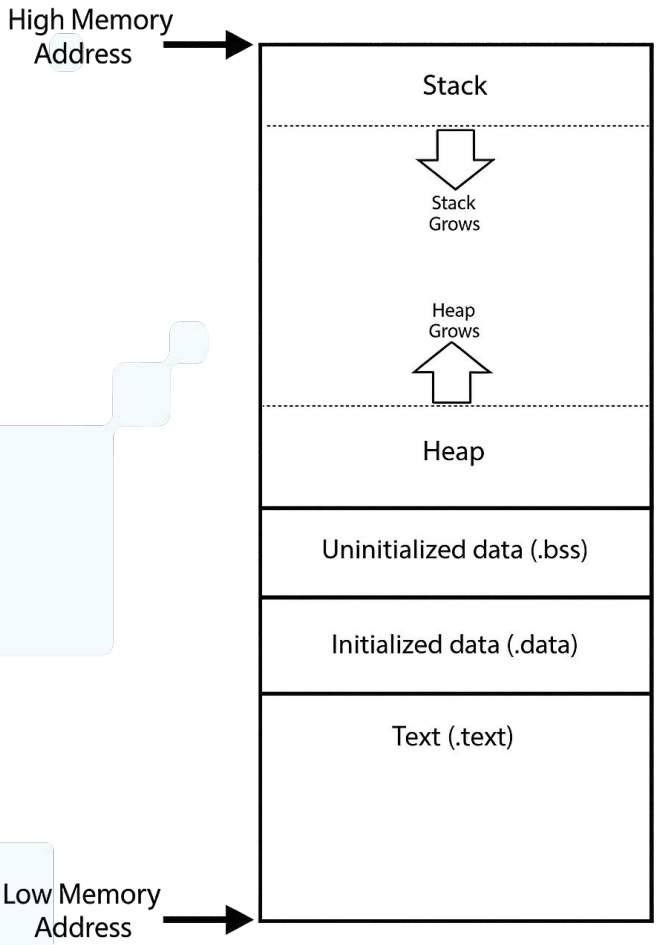
~~**mov** dword ptr [0x0102030405060708], 0x0102030405060708~~

FAST FORWARD

Метки и циклы

```
    .intel_syntax noprefix
    .global main
    .text
main:
    xor eax, eax
    xor ebx, ebx
loop:
    inc eax
    inc ebx
    jmp loop
```

FAST FORWARD



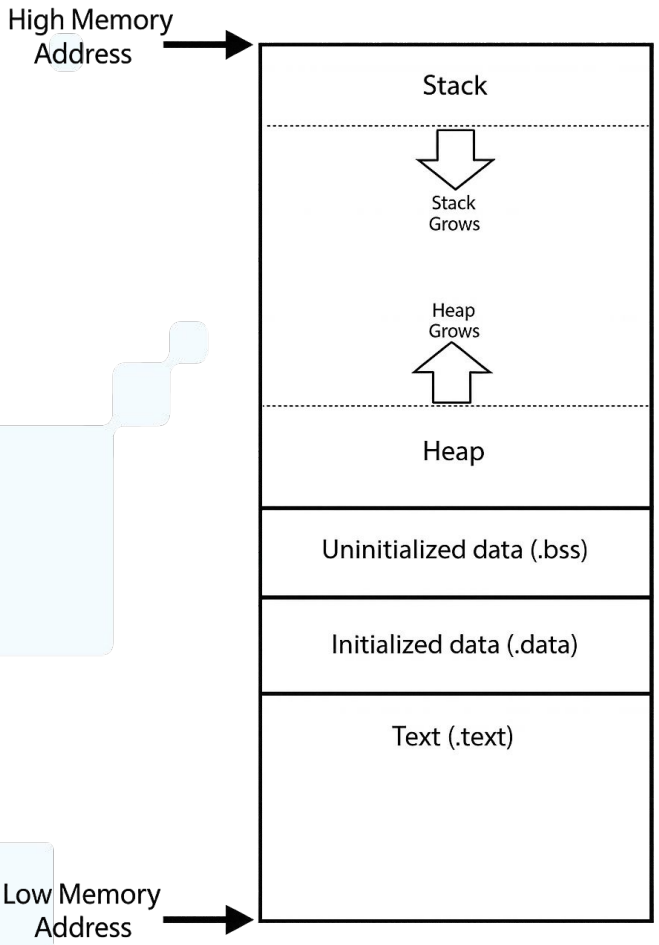
Стек - локальные переменные

Куча - аллокации большого размера

Глобальные переменные

Код

FAST FORWARD



Стек - локальные переменные
RW

RW
Куча - аллокации большого размера

RW
Глобальные переменные

RE
Код

MOV

```
.global main
.text
main:
    mov edi, [rip+x]        8b 3d 0a 00 00 00
    call writei32           e8 09 00 00 00
    call finish             e8 09 00 00 00

x:
    .int 43                 2b 00 00 00
```

FAST FORWARD

MOV

```
.global main
.text
main:
ОШИБКА!    mov [rip+x], edi        48 8b 3d 0a 00 00 00
           call writei32           e8 09 00 00 00
           call finish             e8 09 00 00 00

x:
           .int 43                 2b 00 00 00
```

MOV

```
        .global main
        .text
main:
        mov edi, [rip+x]        8b 3d e1 2e 00 00
        call writei32           e8 09 01 00 00
        call finish             e8 0a 01 00 00

        .data
x:      .int 43                 2b 00 00 00
```

MOV

```
        .global main
        .text
main:
        mov edi, [rip+x]        8b 3d e1 2e 00 00
        call writei32           e8 09 01 00 00
        call finish             e8 0a 01 00 00

        .bss
x:  .int 0                      00 00 00 00
```

FAST FORWARD

Адресация

```
mov eax, [rip + x]        // загрузили 4 байта по адресу x
mov eax, [rip + x + 4]    // загрузили следующие 4 байта

lea rsi, [rip + x]        // (положили в rsi адрес массива x)
mov eax, [rsi + 4]        // косвенная адресация
```

FAST FORWARD

Адресация

BREG + IREG * SCALE + OFFSET

BREG - базовый регистр

IREG - индексный регистр

SCALE - 1, 2, 4 или 8

OFFSET - константа

* RIP только с OFFSET

FAST FORWARD

Адресация

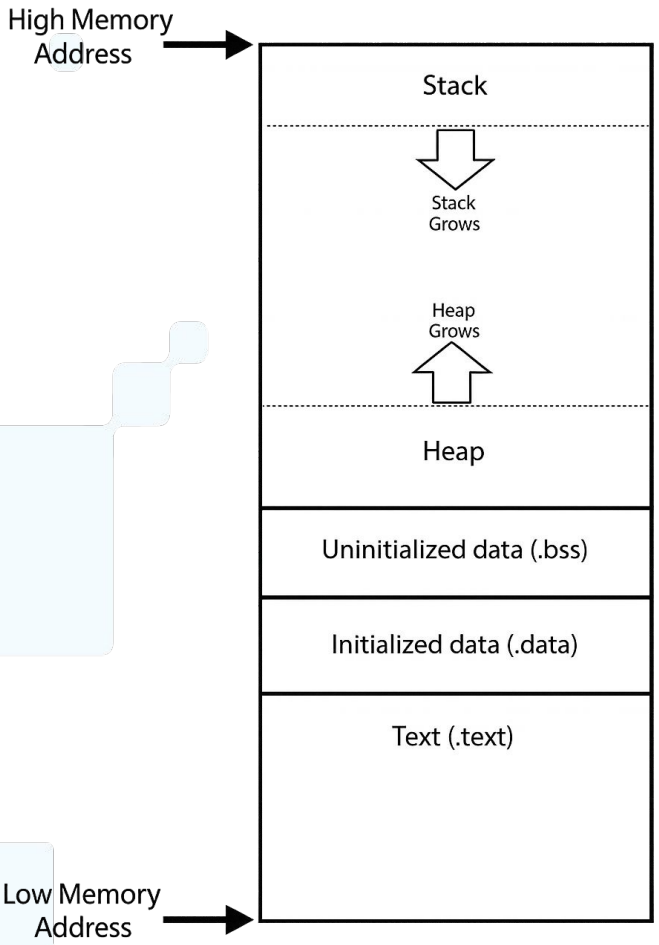
```
mov rdi, 17
lea rsi, [rip + x]
mov eax, [rsi + rdi * 4]


// то же самое
mov eax, [rip + x + 17 * 4]
```

FAST FORWARD

Адресация

```
inc qword ptr [rip+x]
add qword ptr [rsi + rdi * 4], 16
imul qword ptr [rcx + rsi * 8]
// умножает rax знаково, результат в rdx:rax
div qword ptr [rcx + rsi * 8]
// делит rdx:rax беззнаково,
// результат в rax, остаток в rdx
```

FAST FORWARD



Стек - локальные переменные

Куча - аллокации большого размера

Глобальные переменные

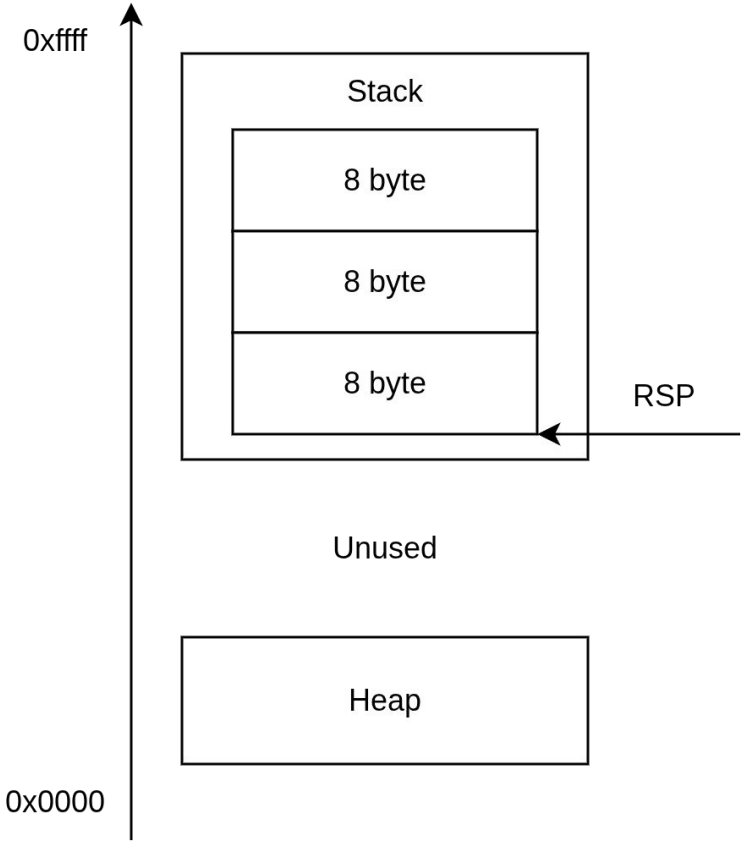
Код

FAST FORWARD

Стек

push rax

pop rax



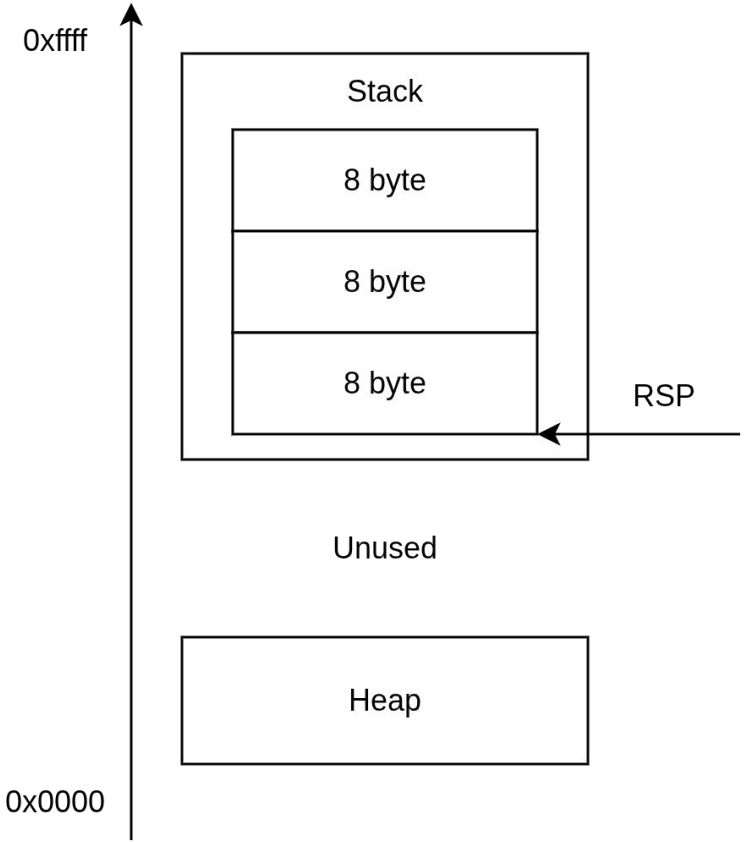
Стек

push rax

```
sub rsp, 8
mov [rsp], rax
```

pop rax

```
mov rax, [rsp]
add rsp, 8
```



FAST FORWARD

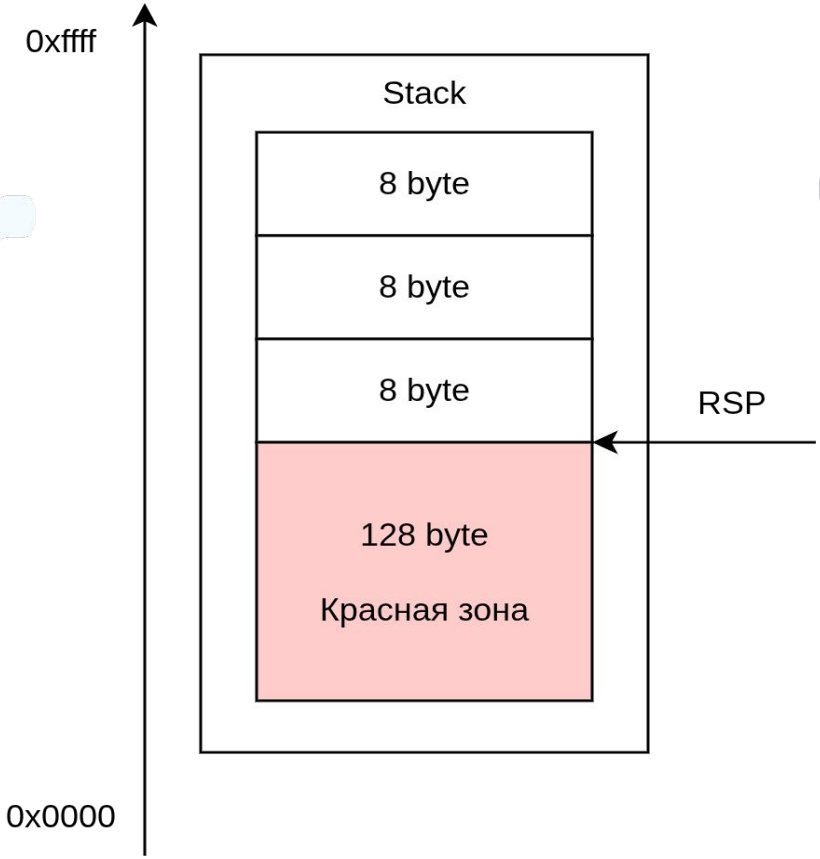
Два обращения к памяти

```
push [rip + x]

sub rsp, 8
mov [rsp], [rip + x]   // Но так нельзя для mov…
```

Стек

Можно писать

в красную зону



FAST FORWARD

Подпрограммы (функции)

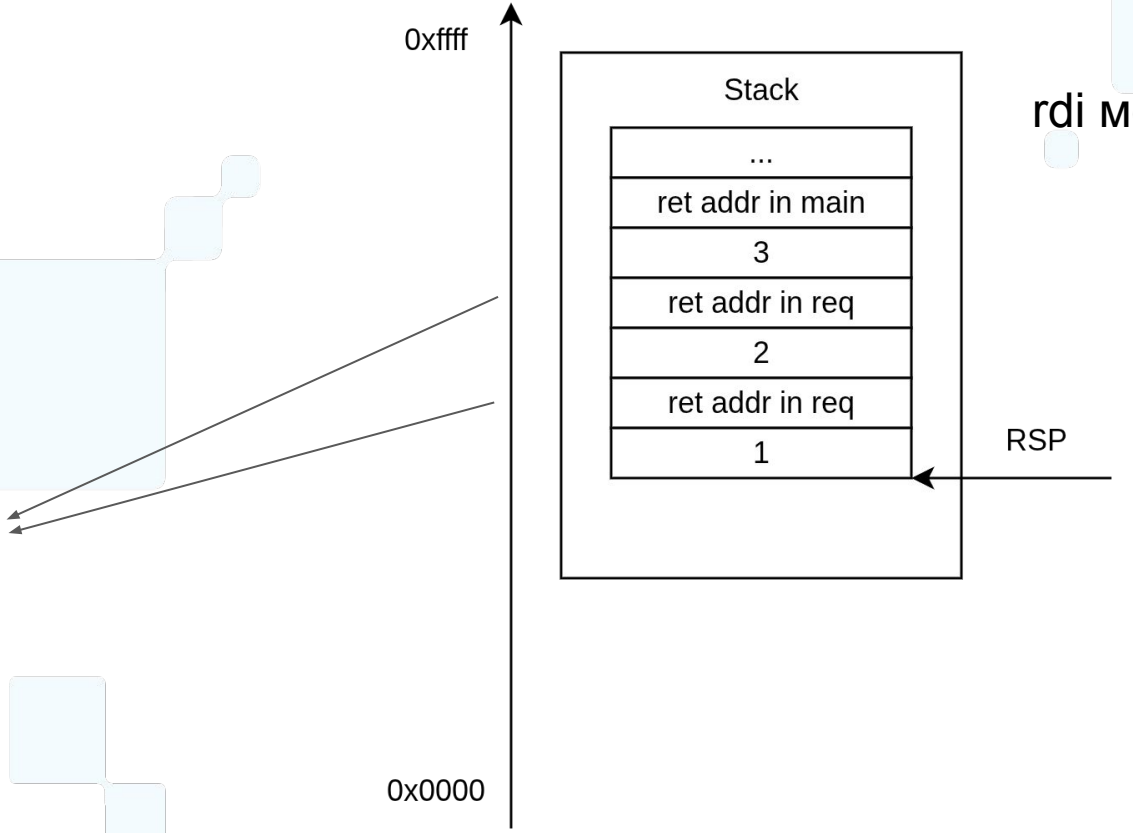
call function == **push** (rip + instruction size) и **jmp function**

function:

ret == **pop** rip

Простая рекурсия

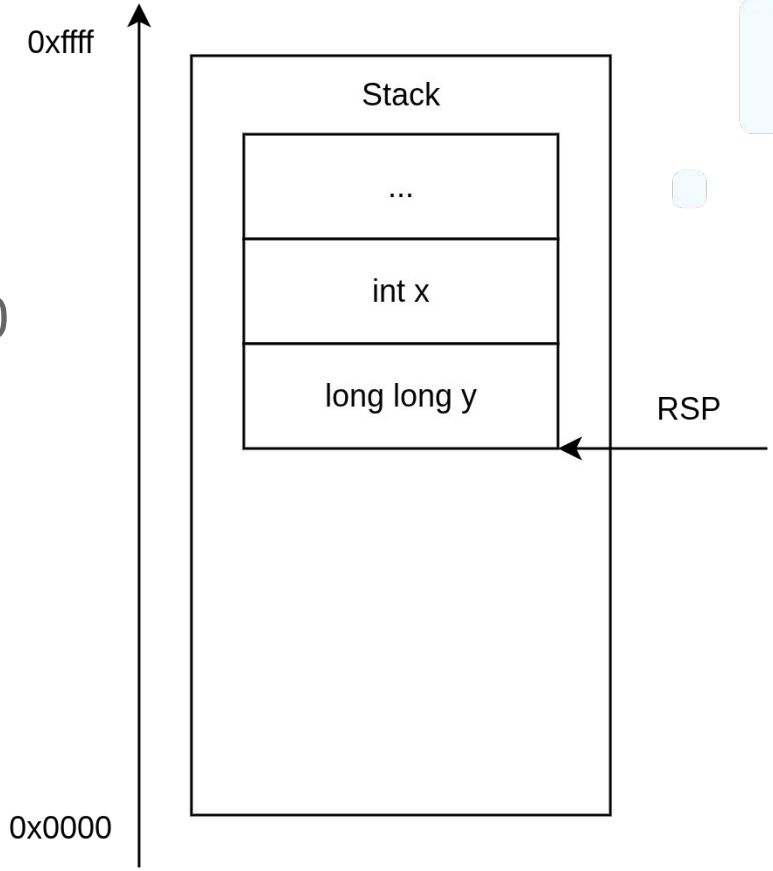
```
.global req
req:
    push rdi
    dec rdi
    jz end
    call req
end:
    add rsp, 8
    ret
```



rdi можно портить

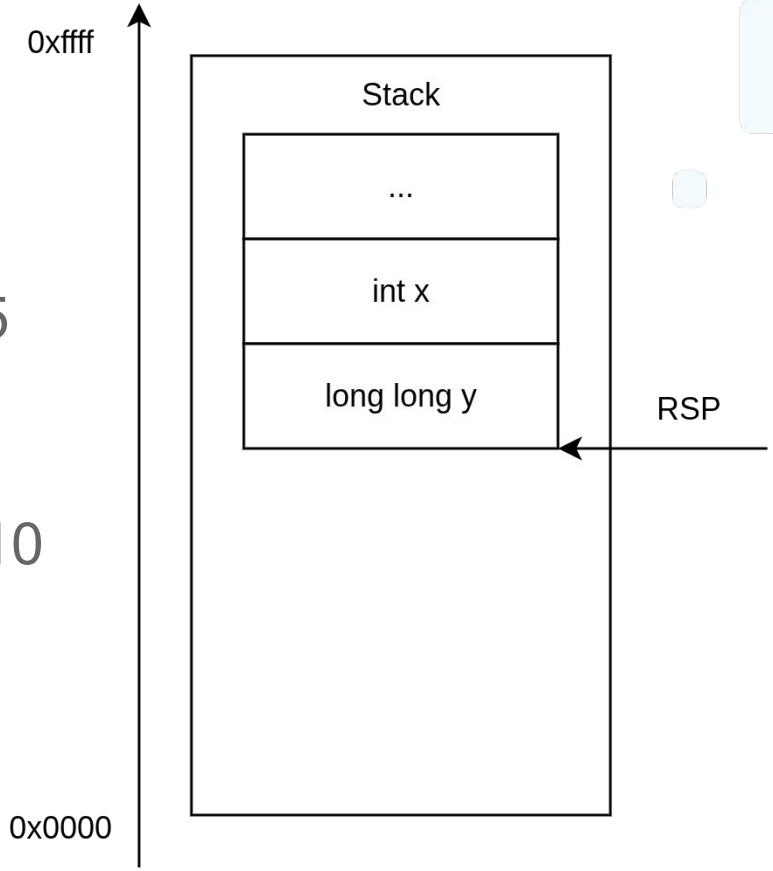
Локальные переменные

```
sub rsp, 16
mov [rsp], 10      // y = 10
mov [rsp+8], 5     // x = 5
add rsp, 16
```



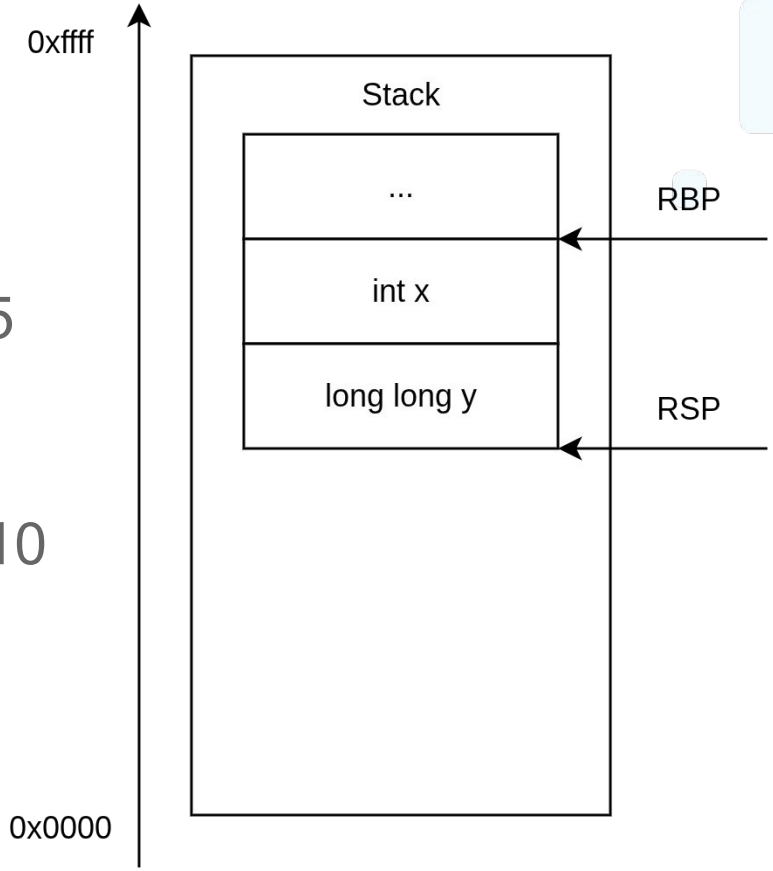
Локальные переменные

```
sub rsp, 8
mov [rsp], 5        // x = 5
sub rsp, 8
mov [rsp+8], 10     // x = 10
add rsp, 16
```



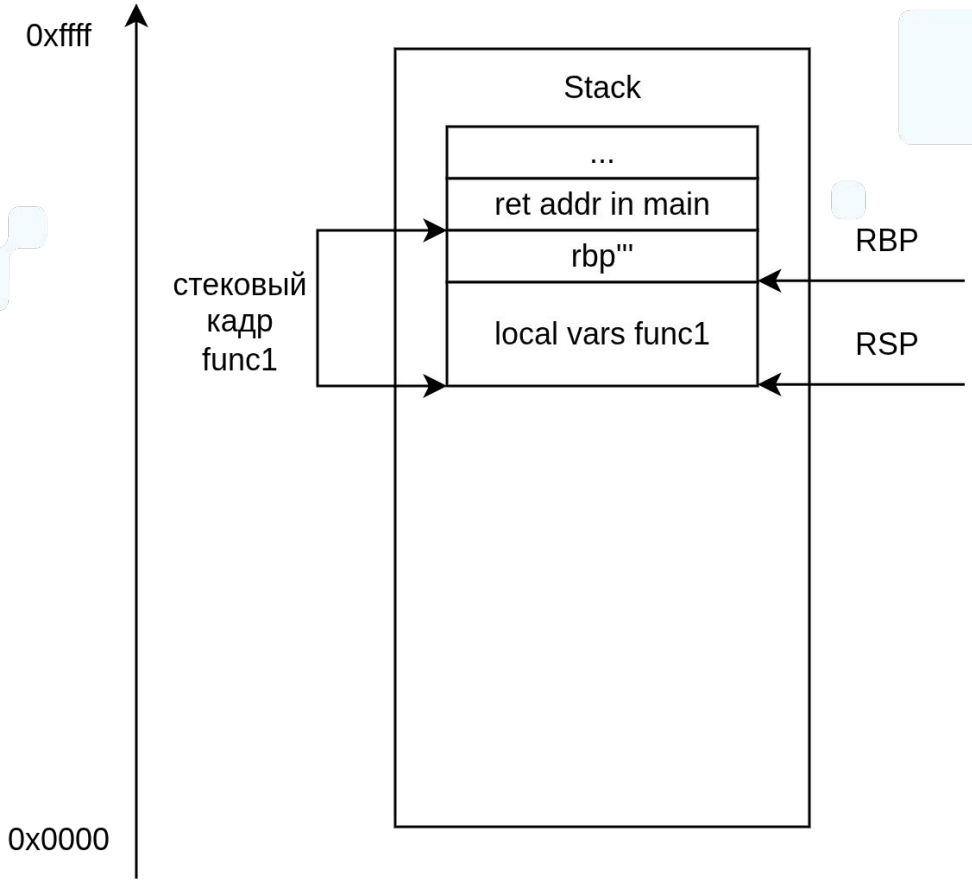
Локальные переменные

```
sub rsp, 8
mov [rbp+8], 5    // x = 5
sub rsp, 8
mov [rbp+8], 10   // x = 10
add rsp, 16
```



Пролог и эпилог

```
func:
    mov rbp, rsp
    …
    mov rsp, rbp
    ret
```



FAST FORWARD

Мотивация договориться

1) Как передавать параметры?
2) Как возвращать результат?
3) Если параметров много - кто очищает стек?
4) Пусть все регистры можно портить подпрограмме…
5) Пусть никакой нельзя…
6) Можно ли Си функцию позвать из Раст?

FAST FORWARD

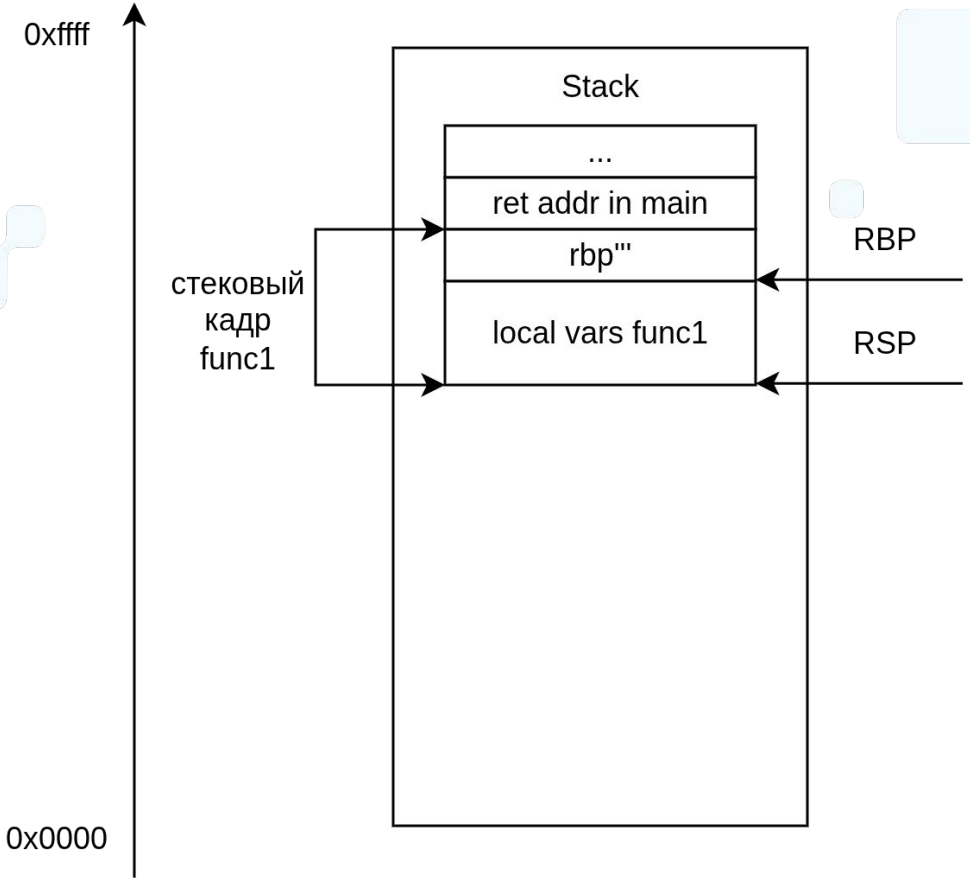
Calling Convention (cdecl)

1) Первые 6 параметров rdi, rsi, rdx, rcx, r8, r9
2) Прочие на стеке по порядку (адреса по порядку)
3) Кто кладёт параметры на стек - тот их и чистит
4) Возвращаем 64 битный и менее результат в rax
5) 128 битный в rdx:rax
6) callee-saved rbx, rsp, rbp, r12, r13, r14, r15
7) caller-saved остальные
8) Перед call rsp кратен 16

FAST FORWARD

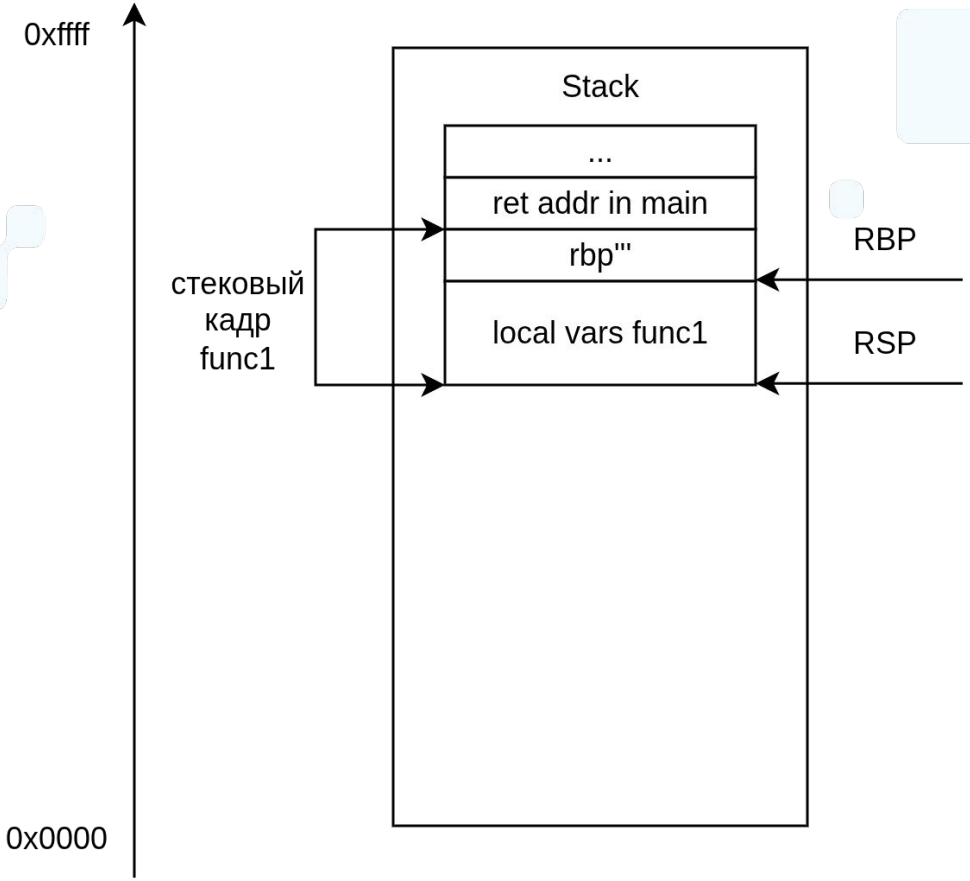
Пролог и эпилог

```
func:
    push rbp
    mov rbp, rsp
    …
    mov rsp, rbp
    pop rbp
    ret
```



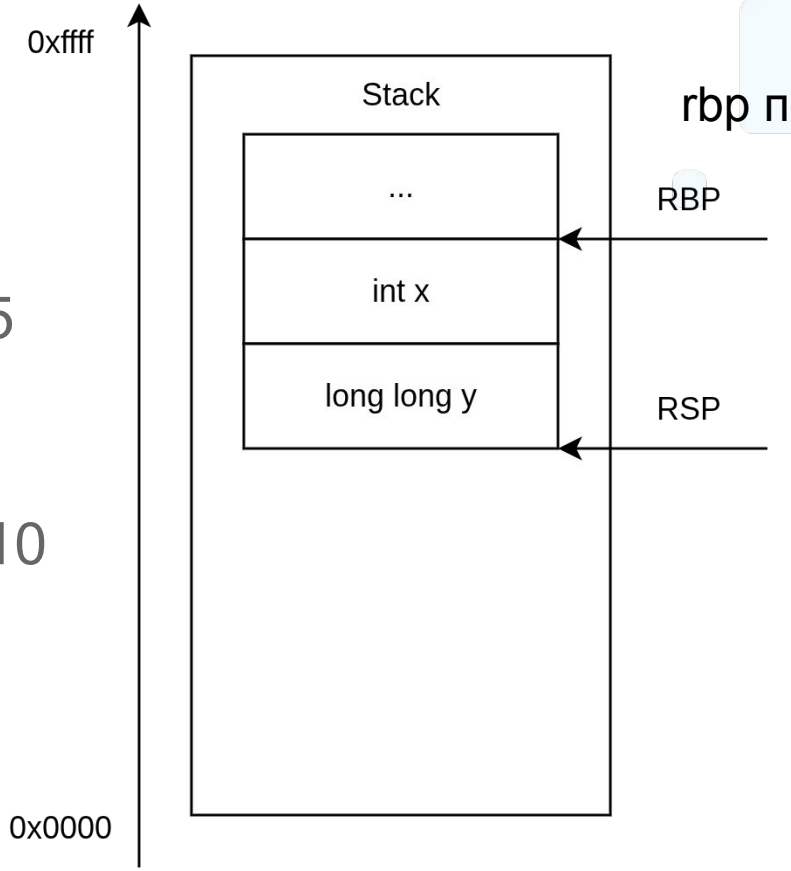
Пролог и эпилог

```
func:
    enter 0, 0
    ...
    leave
    ret
```



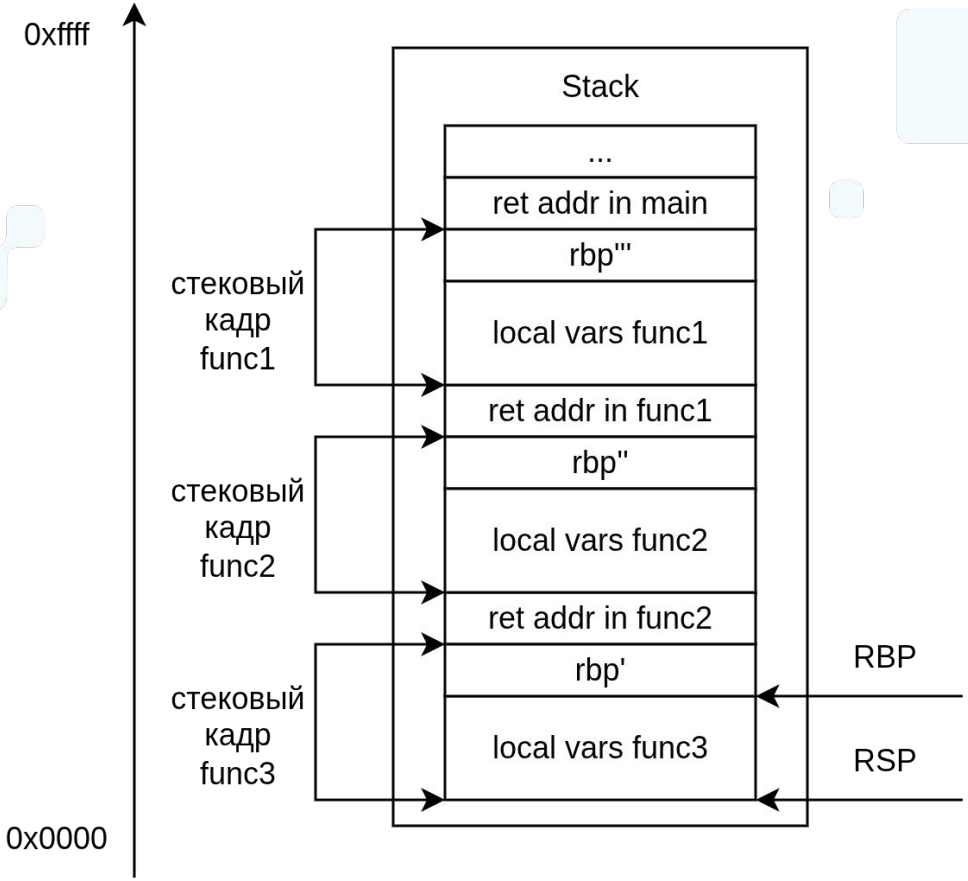
Локальные переменные

```
sub rsp, 8
mov [rbp+8], 5    // x = 5
sub rsp, 8
mov [rbp+8], 10   // x = 10
add rsp, 16
```

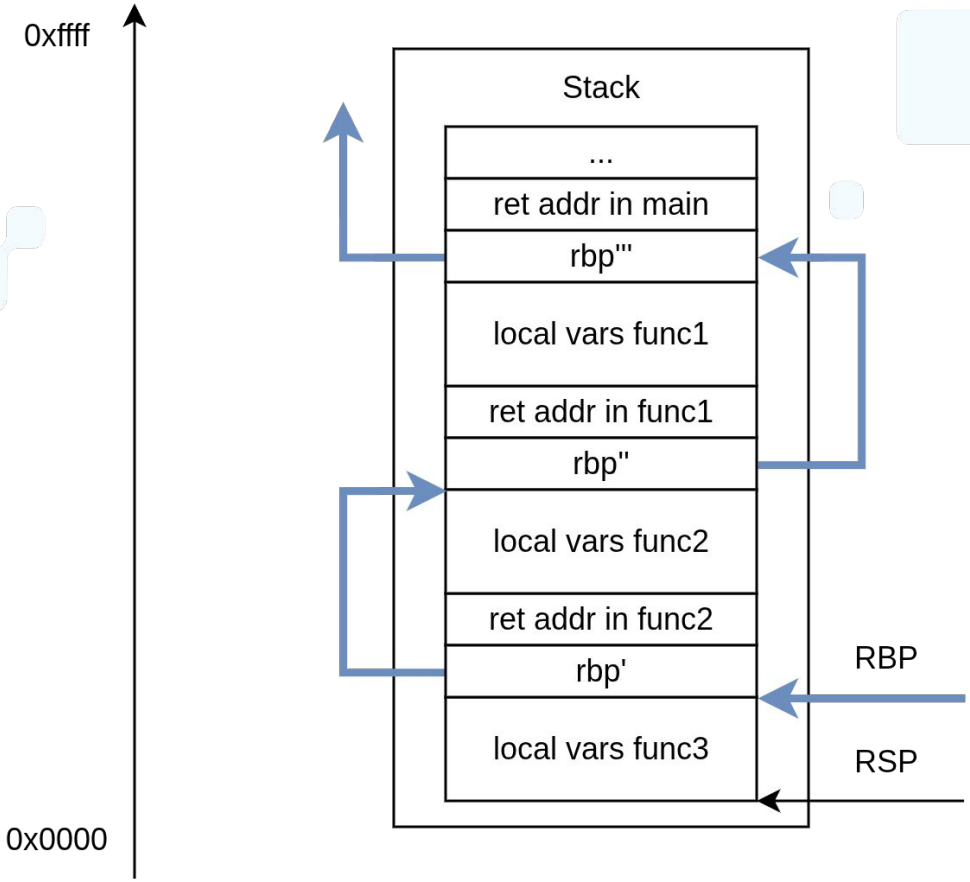


rbp портить нельзя!

Stack frame


0xffff
0x0000
Stack
...
ret addr in main
rbp'''
local vars func1
ret addr in func1
rbp''
local vars func2
ret addr in func2
rbp'
local vars func3
стековый кадр func1
стековый кадр func2
стековый кадр func3
RBP
RSP

Stack frame reverse



FAST FORWARD

Пример

```
int sum(int x, int y) { return x + y; }
.global sum
sum:
    push rbp
    mov rbp, rsp
    mov eax, edi
    add eax, esi
    leave
    ret
```

Вызов C функции

```
int sum(int x, int y) { return x + y; }
.global main
main:
    push rbp
    mov rbp, rsp
    mov edi, 16
    mov esi, 12
    call sum
    leave
    ret
```

FAST FORWARD

“Константы”

```
.byte 0x90          // байт
.skip 5, 0x90       // пропустить

.word 1, 2, 3       // или .short 1, 2, 3 - 16 bit
.int 4, 5, 6        // 32 bit
.quad 7, 8, 9       // 64 bit
```

FAST FORWARD

“Константы”

```
.byte 'H'
.ascii "ello, "      // Строка
.asciz "world!"      // Null-terminated

.byte '\0'
```

Вызов C функции

```
        void print(char * p)

        .global main
main:
        push rbp
        mov rbp, rsp
        lea rdi, [rip + str]
        call print
        leave
        ret

        .data
str:
        .asciz “Hello!”
```

FAST FORWARD

АКОС

QCourse.ru 2026